Enabling Deep Learning on Embedded Systems for IoT Sensor Data Analytics: Opportunities and Challenges

Tharmakulasingam Sirojan, Toan Phung and Eliathamby Ambikairajah
School of Electrical Engineering and Telecommunications
University of New South Wales, Sydney, Australia.
t.sirojan@unsw.edu.au

***Abstract*— Internet of Things (IoT) applications are generating enormous amount of data and demands lower latency responses. Due to the wide-spread deployment of IoT sensors, it is impossible to centralize the data processing to produce real-time responses. Gradually, IoT data analytics are pushed out form the cloud servers and performed in decentralized infrastructures. Usage of embedded hardware platforms are constantly increased to facilitate distributed IoT data analytics. Execution of computation intensive data analytic techniques such as deep learning algorithms need to be optimized based on the constraints of embedded hardware platforms. This paper outlines the opportunities and challenges in enabling deep learning on embedded hardware and summarizes the optimization techniques for deep learning approaches.**

Keywords*— *deep learning, internet of things, embedded systems, sensor data analytics

I. Introduction

In recent years, there has been a significant growth in the data volume produced by the IoT devices. Cisco has estimated that 50 billion IoT nodes will be connected to the internet by 2020 [1]. Centralized cloud-based data analytics are not suitable for the data generated from the large volume of wide-spread IoT sensors. Furthermore, heterogeneity of IoT devices, low latency response requirements and mobility of IoT nodes makes the cloud integration more ambitious. This creates the requirement of new intermediate platforms to facilitate the real-time IoT sensor data analytics. As a solution, fog computing and edge computing paradigms are introduced to provide the computational infrastructure closer to the IoT devices.

This paradigm shift greatly contributes towards the decentralized data analytics. It is not economically feasible to use high performance devices throughout the data analytics hierarchy. Typically, the platforms closer to the IoT nodes are equipped with low power embedded hardware. Fig. 1 outlines the IoT data analytics hierarchy along with the hardware distribution. The choice of hardware and the distribution of data analytics need to be decided based on the application specific requirements and constraints. In real-time (RT) applications, response to an external event need to be produced within milliseconds. Such latency critical data analytics are expected to be performed in the device itself or in an embedded hardware platform which is directly attached to the IoT sensor. This computation platform is defined as edge computation infrastructure as denoted in the lower layer of Fig. 1. The second layer in the hierarchy is named as fog computing infrastructure and it is highly suitable for near real-time (NRT) applications which demands the latency in seconds to fractions of minutes. This fog nodes can be an embedded hardware, networking devices and medium scale

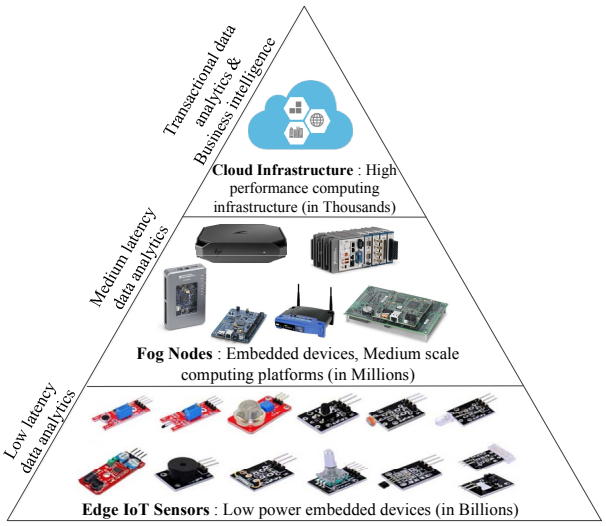


Figure 1. IoT Sensor data analytics hierarchy & hardware distribution

computing platforms that are connected to the IoT nodes via low latency networks such as local area networks (LANs). The top layer represents the cloud infrastructure with high performance computing capability that can be leveraged for transactional and historical data analytics and business intelligence applications.

Nowadays, machine learning based techniques are heavily used in IoT data analytics for pattern recognition, fault identification, condition monitoring, forecasting, etc. Machine learning algorithms statistically learn the knowledge corresponds to the application domain through an input dataset. This process is termed as training. Machine learning models can imitate the functionality of a human brain once they are trained. Execution of tasks using the learned knowledge is referred as inference. Recent advancements such as deep learning in the discipline of machine learning requires huge amount of data set as well as specialized hardware accelerators for training. Different aspects of machine learning algorithms have variable computational requirements.

This paper aims to highlight the challenges and opportunities in the deployment of deep learning algorithms on top of embedded hardware at the edge/fog rather than in a cloud server. Due to this deployment, significant advantages such as lower latency, independence on connectivity and improved data privacy and security can be achieved. On the other hand, most of the embedded devices having stringent constraints on energy, processing power and memory requirement. The effect of each constraint and the

978-1-5386-9418-3/18/$31.00 ©2018 IEEE

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 06:38:44 UTC from IEEE Xplore. Restrictions apply.

improvement opportunities need to be studied carefully to enable an efficient processing of machine learning algorithms on embedded systems.

II. Emerging Opportunities for Embedded Data Processing in IoT

In this IoT era, most of the industrial operations from different applications domains are getting automated through the digital transformation. In such applications, data is produced as well as consumed at the edges of the network. In addition, application requirements and constraints may vary across the domains. Some of the application domains which demands embedded data processing to meet its functional requirements are illustrated below.

A. Computer Vision Applications

In computer vision applications such as autonomous vehicle navigation and robotics, mechanical actuations are triggered based on the decisions made by data analytics. Cameras and other sensors are used to capture the surrounding environment. The captured video stream needs to be processed to identify the surrounding objects, traffic conditions and nearby vehicles [2]. Cloud based data analytics requires continuous video streaming from the vehicle to the cloud. This requires huge amount of network bandwidth. Furthermore, mobility of vehicles can act as a bottleneck to establish a persistent connection between cloud and vehicle. Moreover, the delay in detecting the nearby vehicles can cause severe accidents. In such situations, the expected lower latency could not be achieved with the cloud-based data analytics due to the communications delay. The solution is to provide the data processing infrastructure closer to the vehicle. Since vehicles are subjected to the mobility, the data processing needs to be facilitated with the embedded hardware in the vehicles to achieve the optimal outcome.

B. Smart Home Applications

In smart homes, household devices such as smart bulb, TV and air conditioner are connected to the network for automation. Operations of these devices are determined from the data generated by the sensors that are deployed in the home [3]. The sensors continuously report the data such as such as temperature readings, room occupancy and human activities which are consumed by the devices in the same house. For such scenarios, the cloud computing paradigm is not a best choice since it requires bidirectional data transmission between cloud and home. The edge computation architecture is the exact much for these applications where all the sensor data are processed at an embedded edge gateway in home. This provides better flexibility of device management, privacy protection as well as saves internet bandwidth.

C. Smart Monitoring Applications

IoT is heavily used in smart monitoring applications to digitalize the physical world into machine readable data. Resource constrained sensors and embedded data acquisition systems (DAQ) are used to sense the environmental conditions. The captured data is leveraged by the systems such as natural disaster monitoring, habitat monitoring, faulty condition monitoring, etc. Each monitoring applications requires different types of environmental conditions at different time resolutions. The time resolution of data plays a critical role in these applications since it directly contributes to the generated data size. Higher sampling rates produce huge amount of data in real-time [4]. In addition, some monitoring scenarios such as natural disaster monitoring requires low latency data analytics. Furthermore, such monitoring applications require the correlation of geographically distributed sensor data. The data processing infrastructure also need to be distributed, in contrast to the centralized cloud, to process the data locally and extract out the key information such as abnormalities. This deployment facilitates the key feature transformation between adjacent nodes to detect an event rather than sending the huge amount of raw data to the cloud. Typically, resource constrained embedded hardware is attached with the DAQ systems for local data processing.

Thus, embedded data processing at the network edges can address the challenges introduced by the state of the art IoT applications from different domains. Lower latency [5], bandwidth efficiency, privacy protection, mobility support and geographical distribution are the key outcomes that can be gained from decentralized embedded data processing.

III. Overview of Deep Learning Techniques & Computational Requirements

Deep learning is the subset of a broad field of machine learning. The machine learning approaches are adapted from the functional mechanisms of a human brain. Neural networks are the most popular and widely used machine learning approach which is derived from connecting the main computational element of a human brain called neuron. Each connection to the neurons has a scaling factor defined as weight, to scale the input signal to a neuron. Neuron is capable to produce the output from the input signal through the activation function associated with it. Typically, the neurons are arranged as layers to form a neural network. Nowadays, more computationally complex structures are formulated based on the aforementioned building blocks. This creates a new discipline under machine learning called deep learning.

Deep learning approaches are becoming more prominent solutions for many artificial intelligence (AI) applications such as image recognition, robotics and speech recognition since it yields state-of-the-art accuracy. In contrast to the traditional neural networks, deep neural networks can extract higher level features form raw input and it consists a greater number of layers varies from five up to thousands. In addition, the layer arrangements and configurations

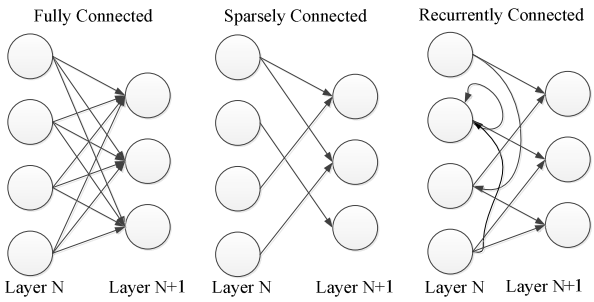


Figure 2. Types of connections in deep neural networks

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 06:38:44 UTC from IEEE Xplore. Restrictions apply.

significantly vary from the conventional neural networks. There are three main types of connections that are widely used in the deep learning architectures: fully connected, sparsely connected and recurrently connected as shown in Fig. 2. In fully connected layers, all input activations contribute towards the output. In contrast, sparsely connected layers only consider a small neighborhood in the input, which is known as receptive field. Convolutional layer is the widely used sparsely connected layer in deep learning architectures. Recurrent connections are commonly used in the networks that have internal memory to store the intermediate results for later processing. In addition, there are some common functionalities such as pooling and normalization that have been frequently used in deep learning. Pooling is used for down sampling or dimensionality reduction. Different normalization techniques are used to improve the learnability of the network as well as the training speed.

All these implementations require matrix manipulation since the input data for a network is commonly represented as matrix with varied dimensionalities. Multiplications and additions are the basic operations to achieve weighted sums of input matrix. Popular deep learning structures such as AlexNet [6], GoogLeNet [7] and ResNet [8] requires 724 million, 1.43 billion and 3.9 billion arithmetic operations respectively, to produce an output. It is not feasible to provide such computational requirements with embedded devices. In early days, deep learning structures are developed with the objective of maximizing the task accuracy rather than considering the implementation complexity. Recent researches insisted that the deep learning models need to be designed to maximize accuracy while minimizing the energy requirement and computational complexity in order to increase the usability of deep learning approaches into practical applications.

IV. Challenges of Real-time Data Processing in Embedded Hardware

This section outlines the challenges that need to be addressed during the deployment of deep learning based approaches in embedded systems to achieve real-time sensor data analytics.

A. Programmability

In cloud-based data analytics, programmers can use scalable and reliable machine learning services and tools from the cloud service providers such as Amazon and Microsoft Azure. The cloud provider handles the deployment and runtime executions with the appropriate hardware to meet the expected functional requirements of the deployed application. However, implementation of deep learning approaches in embedded hardware is more challenging due to these embedded nodes are most likely heterogeneous platforms. Because of this heterogeneity, it is hard to find predefined software libraries and tools support for these platforms. In most cases, the programmer needs to write each data processing functions from the scratch. This is time consuming as well as tedious task since all the functionalities and task synchronizations need to be optimized based on the capabilities of the hardware. These additional workloads make the programmability difficult in the embedded hardware compared to the cloud infrastructure.

B. Memory

Memory is one of the limited resources in embedded hardware that need to be efficiently used. Typically, memory is designed as hierarchy to compensate the speed gap between the processors and memory. Lower levels such as registers, and caches provide high speed data access but extremely limited in availability. Higher levels such as solid-state disks cater greater storage capacity than the lower order memories. However, higher level memories consume more energy than small memories. Furthermore, additional data transfers among the memory hierarchy negatively affects the energy consumption. An optimized dataflow needs to be derived to minimize access from the higher energy consuming memory levels. In real-time sensor data analytics, the captured data, weights of the deep learning model and the intermediate results need to be fit in the lower order memories simultaneously to speed up the data processing as well as to reduce the energy consumption. In most of the applications that samples the input with higher sampling rates, the raw data need to be discarded soon after the processing and only the key information is transferred to the higher-level persistent storage or cloud. Due to the limitations of the on-chip memories, the storage complexity of deep leaning models needs to be improved.

C. Processing power

Typically, Application-Specific Integrated Circuits (ASIC), Field Programmable Gate Arrays (FPGA) and microprocessors are used for data processing in embedded hardware. Modern ASICs contains microprocessors and layers of memory blocks and commonly known as System-on-Chip (SoC). FPGAs are parallel structures that contains an array of logic blocks, slice registers, look-up tables Random Access Memory (RAM) blocks and Digital Signal Processor (DSP) slices. DSPs are specialized microprocessors that are optimized for signal processing tasks. Furthermore, FPGA hardware that contains DSP slices facilitate high speed data acquisition and processing. However, the embedded hardware poses severe constraints on these resources due to the resource scarcity.

In real-time sensor data analytics, DAQ system continuously samples the data with the configured sampling rate. Incoming data stream need to be processed as early as possible to free-up the memory for the next data sequence. Generally, real-time data processing is executed as batches and each batch is termed as data frame. The time limit to process a data frame is decided by the number of data points in a frame as shown in Fig. 3. Boosting the number of data points in a frame extends the processing time, but it

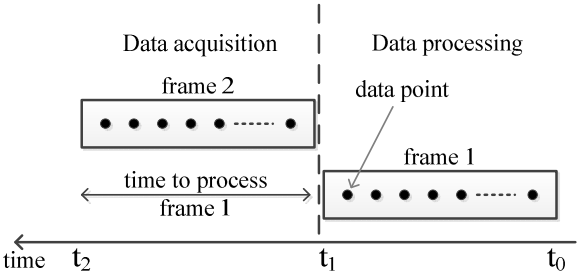


Figure 3. Time constraints on data frame processing.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 06:38:44 UTC from IEEE Xplore. Restrictions apply.

increases the computational complexity of a frame. Timing violations in data processing will typically result in data overwriting and data losses. Several optimization techniques such as parallelized and pipelined execution of tasks can be used to achieve the required timing. However, all these optimization techniques are limited with the available hardware resources.

Thus, real-time sensor data analytics in embedded hardware platforms create conflicting requirements such as minimizing memory usage and processor usage while ensuring that all the timing deadlines need to be met at run-time. This enforces that the deployed functionalities need to be optimized in terms of memory usage and processing power requirement.

V. Possible Optimizations for Deep Learning in Embedded Hardware

This section focuses on the optimizations of deep learning models for embedded hardware. Typically, deep learning models are trained in the machines with highly parallelizable hardware such as Graphical Processing Unit (GPU) or cloud infrastructures. After training, these models might be deployed in the embedded devices to analyze the data based on the learned weights. In such situations, the models need to be optimized based on the embedded hardware and application requirements. Some optimization techniques, that can be used for deep learning models to enable real-time inference in resource constrained devices, are discussed below.

A. Weight pruning and compression

The concept of weight pruning is to remove weights of a deep learning model that are lower than a threshold value. Accuracy of the trained model get affected by this operation. In order to compensate the accuracy, the model is re-trained to fine tune the remaining weights. This process can be repeated until the desired accuracy as well as weight reduction is achieved. The pruned deep learning model has fewer valid weights since the pruned weights are set to zero. This reduces the number of computations required during the inference.

In addition, compression techniques can be applied to the weights after pruning to reduce the required storage. Compressed sparse raw (CSR) format and compressed sparse column (CSC) format are the successful techniques in weight compression. These techniques take advantage of the sparse nature and can achieve around 20-30% of memory space reduction [9].

B. Filter decomposition

Filter decomposition can be applied to optimize the convolutional neural networks in terms of computational complexity and memory requirement. Typically, convolutional layers use filters to extract features from the raw input. The concept of filter decomposition is to substitute a larger filter with a series of smaller filters and apply them sequentially. It is based on the associativity of the convolution operation. For instance, a M x N filter in a convolutional layer requires M x N multiplications for each input. When this filter is decomposed in to M x 1 and 1 x N filters, required multiplications is reduced to M + N. This decomposition can cover the same receptive filed, that covered by M x N filter, with fewer number of weights and multiplications. Similarly, higher dimensional convolutions can be decomposed into a set of lower order convolutions to reduce the required parameters as well as computational cost [10].

C. FFT based convolutions

Fast Fourier Transformation (FFT) based convolutions can reduce the computational complexity of the convolutional layers, especially when its filter sizes are large. Typically, convolution operation is mapped to the matrix multiplication in computing platforms. In GPUs, these matrix multiplications are optimized by the software libraries such as cuDNN. However, the embedded hardware does not support such optimizations. In such situations, FFT based convolutions can be used to reduce the required number of multiplications from $O(PQMN)$ to $O(PQ \log PQ)$ where input size is P x Q and filter size is M x N [11]. Convolution operation can be achieved through the multiplication of filter with the input feature map in the frequency domain. Then inverse FFT is used to regain the time domain. These FFT based computations can be efficiently performed in DSP slices that are commonly available in embedded hardware.

D. Knowledge distillation

The idea of knowledge distillation is to transfer the knowledge learned by the complex model to a simple model [12]. Complex deep learning models cannot be deployed in embedded systems due to the constraints on resources. On the other hand, the learnability of simple and shallow networks is extremely poor. Knowledge distillation technique recommends that the predicted score from the complex models can be used to train the simple shallow models. In such trainings, simple models can achieve higher accuracy than the training with the actual class labels from the dataset.

E. Quantization

Quantization techniques can be applied to the deep learning model parameters in order to reduce the computational complexity as well as storage requirement. Quantization is a process of mapping the data to a smaller set of intervals. Having smaller number of quantization levels reduce the complexity as well as memory, but it negatively affects the accuracy. A good balance needs to be maintained between the accuracy and resource consumption. Furthermore, different data types, that require lower number of bits, can be used to represent the model parameters [13].

VI. Conclusion

Cloud computing based centralized data analytics is not the best choice any more due to the emerging requirements of IoT. Data processing need to be moved closer to the sensors to facilitate the real-time applications. In such situations, data processing algorithms must be implemented in embedded hardware platforms. Recently, deep learning algorithms achieved state-of-the-art accuracy in data analytics tasks from different domains. However, deployment of these algorithms is challenging due to its high resource requirement. This paper summarizes the

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 06:38:44 UTC from IEEE Xplore. Restrictions apply.

opportunities and challenges in enabling deep learning based approaches in embedded hardware and highlights some optimization techniques that can be used for real-time IoT applications.

REFERENCES

[1] A. Sherry, "Fog Computing and the Internet of Things: Extend the Cloud to Where the Things Are," Cisco Systems, April 2015.

[2] C. Chen, A. Seff, A. Kornhauser, and J. Xiao, "Deepdriving: Learning affordance for direct perception in autonomous driving," in *Proceedings of 15th IEEE International Conference on Computer Vision* (ICCV), 2015.

[3] P. Verma and S. K. Sood, "Fog Assisted-IoT Enabled Patient Health Monitoring in Smart Homes," in *IEEE Internet of Things Journal*, 2018.

[4] R. Song, S. Lu, T. Sirojan, B. T. Phung and E. Ambikairajah, "Power quality monitoring of single-wire-earth-return distribution feeders," *International Conference on High Voltage Engineering and Power Systems (ICHVEPS)*, 2017.

[5] T. Sirojan, S. Lu, B. T. Phung, D. Zhang and E. Ambikairajah, "Sustainable Deep Learning at Grid Edge for Real-time High Impedance Fault Detection," in *IEEE Transactions on Sustainable Computing*. doi: 10.1109/TSUSC.2018.2879960

[6] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks". in *Proceedings of NIPS*, 2012.

[7] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan and V. Vanhoucke . "Going deeper with convolutions." in *IEEE conference on computer vision and pattern recognition*. 2015.

[8] K.He, X. Zhang, S. Ren, J. Sun, "Deep residual learning for image recognition." in *IEEE conference on computer vision and pattern recognition*. 2016.

[9] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," in *Proceedings of ICLR*, 2016.

[10] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proceedings of CVPR*, 2017.

[11] C. Dubout and F. Fleuret, "Exact acceleration of linear object detectors," in *Proceedings of ECCV*, 2012

[12] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network,"in *Deep Learning Workshop NIPS,* 2015.

[13] G. Venkatesh, E. Nurvitadhi, and D. Marr, "Accelerating deep convolutional networks using low-precision and sparsity," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 06:38:44 UTC from IEEE Xplore. Restrictions apply.